Modelo de Arquitetura do Sistema

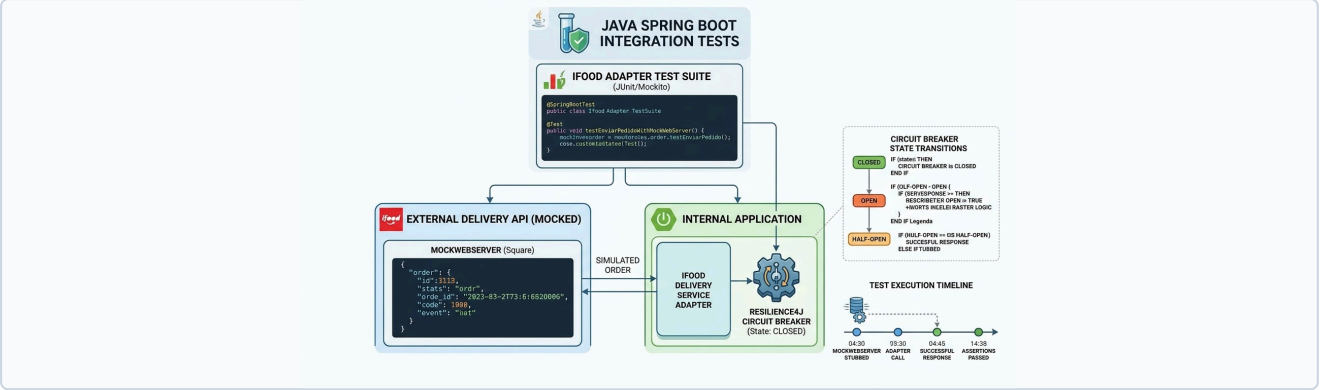
PLATAFORMA MULTILOJAS, HUB DE ENTREGAS, ORQUESTRAÇÃO IA & INFRAESTRUTURA RESILIENTE

1. Arquitetura Geral & Visão Holística

O sistema baseia-se numa arquitetura em camadas orientada a serviços com backend em **Java Spring Boot**. O núcleo é responsável por gerir o ciclo de vida de pedidos, calcular fretes determinísticos e integrar adaptadores de plataformas de entrega (iFood, Uber, Keeta, 99 e Entrega Própria). Agentes de IA operam como camada consultiva desacoplada.

2. Padrão de Integração, Resiliência e Testes

Abaixo detalha-se o modelo de isolamento de testes e resiliência de rede do adaptador externo (exemplo iFood), garantindo tolerância a falhas através do padrão Circuit Breaker (Resilience4j) e validação isolada com MockWebServer.



Detalhamento do Componente de Resiliência:

- **Suite de Testes de Integração:** Executada com `@SpringBootTest`, JUnit 5 e MockMockito, simulando respostas HTTP reais sem depender das APIs de produção.
- **MockWebServer (Square):** Substitui as APIs externas de entrega em ambiente de teste, devolvendo payloads JSON estubados para simular cotações, aceitações e eventos webhook.
- **Resilience4j Circuit Breaker:** Mantém o estado `CLOSED` em operação normal. Transita para `OPEN` após falhas consecutivas do fornecedor, ativando rotas alternativas ou *fallback* automático sem bloquear as threads do sistema principal.

3. Hub Multilojas e Matriz de Provedores de Entrega

O checkout dinâmico avalia a localização do cliente e a configuração de cada restaurante, calculando o valor final do frete e estimativa de entrega (ETA). Nenhuma contratação é executada sem a confirmação explícita do cliente.

Plataforma	Modalidade	Capacidades Oficiais	Mecanismo de Fallback
iFood	API Oficial Merchant	Cotação em tempo real, envio de pedido, webhooks de estado, cancelamento.	Desativação da modalidade ou transição para Entrega Própria.
Uber Direct	API Logistics	Cotação por coordenadas, rastreio dinâmico, código de validação na entrega.	Ocultação da opção na listagem do checkout.
Keeta / 99	API Parceiros	Cotação por raio de cobertura, cálculo de ETA baseada em frota.	Notificação ao cliente e opção de recolha no local.
Entrega Própria	Motor Interno	Definição manual de taxas por bairro/km, raio fixo e tempo estimado estático.	Sempre disponível caso as APIs terceiras falhem.

4. Regras Determinísticas de Checkout e Carrinho

Para garantir a consistência das transações e evitar inconformidades nos pedidos, o backend impõe validações estritas no fluxo de compra:

- **Isolamento de Carrinho por Loja:** É estritamente proibido misturar artigos de estabelecimentos distintos no mesmo carrinho. Ao alterar de loja, o utilizador é solicitado a confirmar a limpeza do carrinho ativo.
- **Cotações Temporizadas e Idempotência:** As cotações de entrega possuem uma validade máxima de 5 minutos. Após o período de expiração, o backend força a revalidação do frete. A criação de pedidos utiliza chaves de idempotência (`X-Idempotency-Key`) para prevenir cobranças duplicadas.
- **Opção de Recolha (Takeaway):** Disponível por padrão para todos os estabelecimentos, eliminando taxas de frete do cálculo final.

> **Garantia de Integridade Financeira:** Os cálculos de valor final, elegibilidade, taxas e autorização de pagamento são 100% determinísticos e executados exclusivamente pelo backend Spring Boot. Nenhum modelo de IA tem autorização para alterar preços ou aprovar transações.

5. Arquitetura de IA: CrewAI, MCP e Serviços Consultivos

A camada de inteligência artificial atua exclusivamente em tarefas consultivas, assistindo o cliente na comparação de opções de pratos, sugestões de menu e explicabilidade de prazos de entrega.

Orquestração com CrewAI

Agentes especializados operam em fluxos estruturados (ex: *MenuAdvisorAgent* e *DeliveryComparatorAgent*). Os agentes analisam os dados já validados pelas APIs REST e fornecem recomendações personalizadas ao cliente.

Protocolo MCP (Model Context Protocol)

Atua como ponte segura entre o servidor de linguagem e as ferramentas do backend. Expõe funções somente-leitura (ex: `get_restaurant_menu`, `compare_delivery_options`) sem acesso direto à base de dados.

6. Infraestrutura Hardware, GPU/CUDA e Estratégia de Fallback

O ambiente de inferência local foi desenhado para maximizar a utilização de recursos acelerados por hardware, garantindo simultaneamente resiliência em cenários de saturação ou falha de componentes.

- **Ambiente Host GPU:** Servidor equipado com GPU NVIDIA RTX 3050 (8 GB VRAM), executando motor Ollama nativo com aceleração CUDA para LLMs de menor dimensão (ex: Llama 3 8B Quantizado).
- **Isolamento de Máquinas Virtuais (Multipass):** Como as instâncias VM/Docker da aplicação não possuem *GPU passthrough* direto, a comunicação com o Ollama é efetuada através de endpoint HTTP interno protegido na rede do host.
- **Gestão de Concorrência e VRAM:** Limites estritos de contexto (`num_ctx: 4096`) e filas com *backpressure* impedem cenários de erro por falta de memória (OOM).

> **Plano de Contingência (CPU Fallback):** O sistema de compras e entregas permanece 100% operacional mesmo na ausência completa de GPU ou indisponibilidade do motor de IA. Nesses casos, o sistema desativa a interface conversacional e mantém a navegação tradicional por catálogo sem impacto no processamento de pedidos.